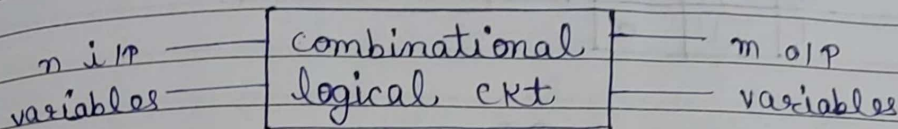


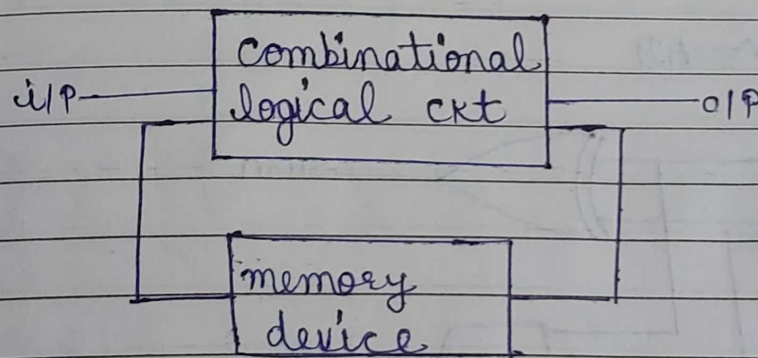
Unit - 3 Combinational Logic circuit design

Combinational circuit :-



Sequential circuit :-

The values of o/p variables not only depends on the present i/p variables but also on the previous history of i/p, which are stored & retained in the memory device like flip-flop, registers, counters etc.



Half adder :-

The ckt that adds two bits is called half adder. The truth table is given below.

i/p		o/p	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

K-map for sum:

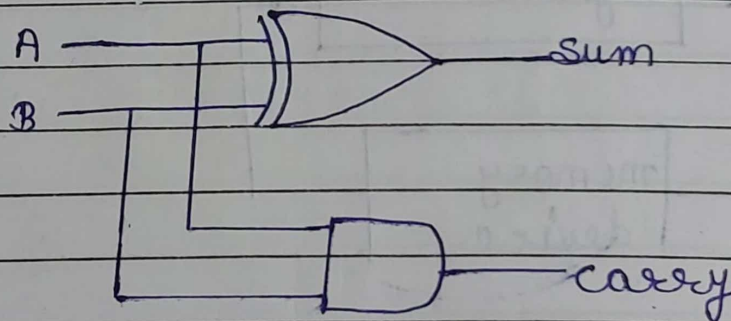
A \ B	0	1
0	0	1
1	1	0

$$Y = A\bar{B} + \bar{A}B = A \oplus B$$

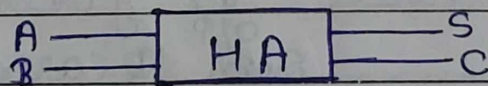
K-map for carry:

A \ B	0	1
0	0	0
1	0	1

$$Y = AB$$



Ckt diagram:

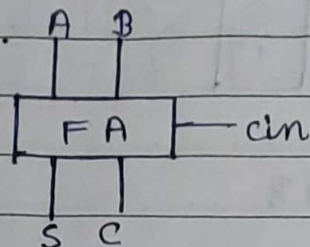


Full adder:

A ckt that adds three bits is called

Spiral

full adder. Here i/p are A, B, cin.



Input			output	
A	B	cin	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

K-map for S:

A \ B	cin			
	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$Y = A\bar{B}\bar{C} + \bar{A}\bar{B}C + ABC + \bar{A}B\bar{C}$$

$$Y = \bar{C}(\bar{A}\bar{B} + AB) + C(\bar{A}B + A\bar{B})$$

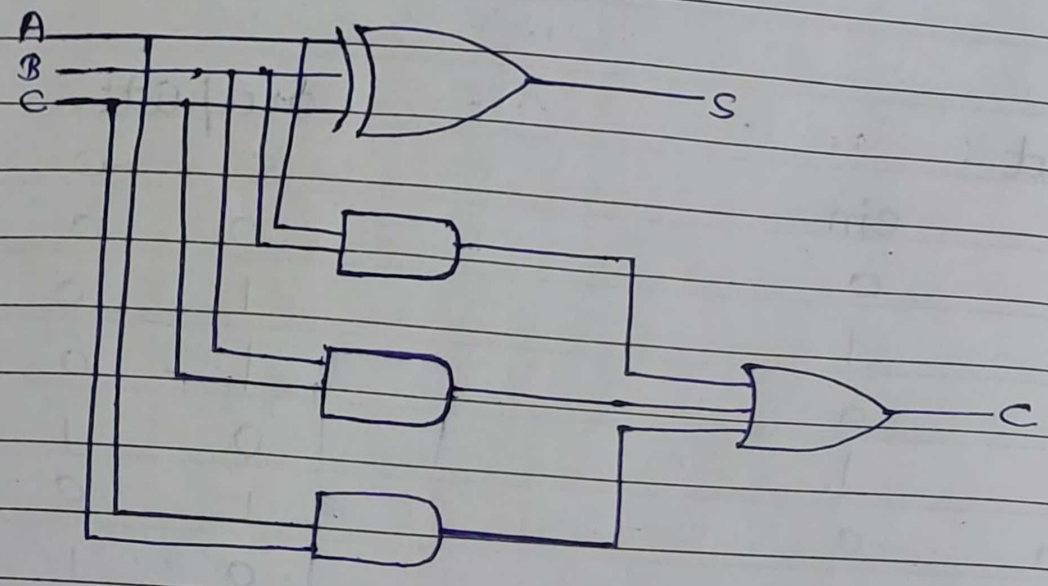
$$Y = \bar{C}(A \oplus B) + C(\overline{A \oplus B})$$

$$Y = \text{cin} \oplus A \oplus B$$

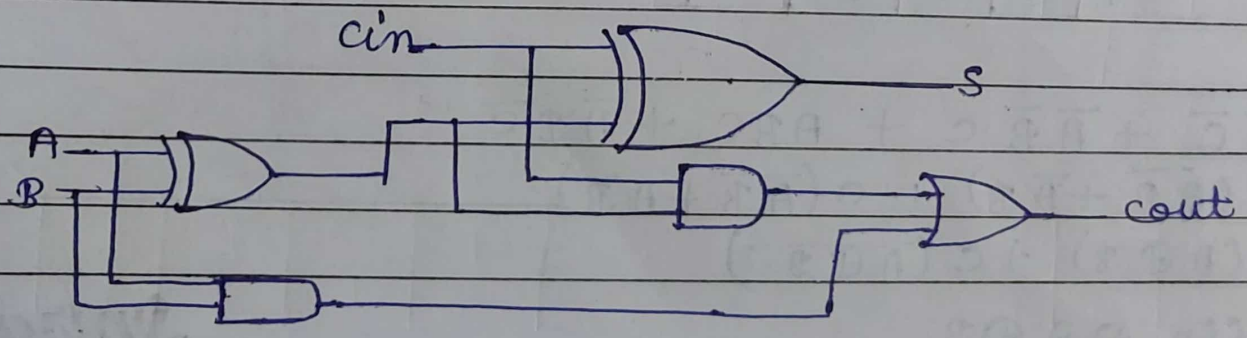
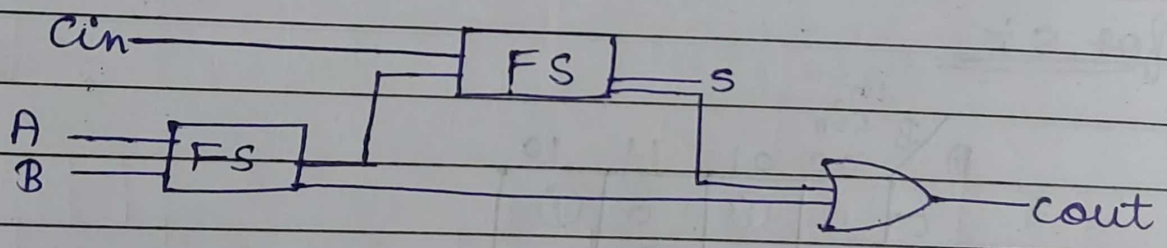
K-map for C :-

		BC			
A		00	01	11	10
0		0	0	1	0
1		0	1	1	1

$$Cout = AC + BC + AB$$



Full adder using two half adder :-



Half subtractor :-

It is a combinational ckt which subtract 2 bits and produced their difference D and borrow out (Bout)

Truth table :-

Input		Output	
Miniend	Subtractor	Diff.	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

K-map for diff.

x \ y	0	1
0	0	1
1	1	0

K-map for Bout

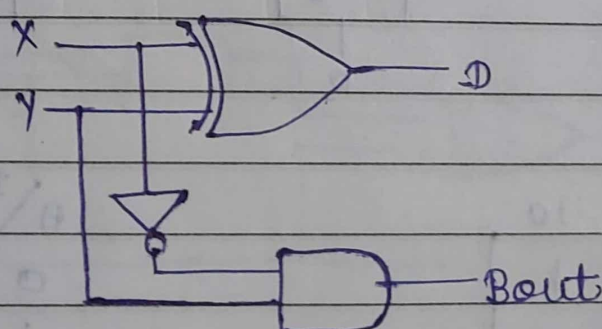
x \ y	0	1
0	0	1
1	0	0

$$D = \bar{X}Y + X\bar{Y}$$

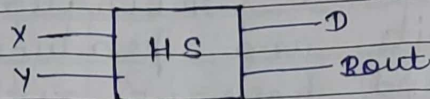
$$D = X \oplus Y$$

$$Bout = \bar{X}Y$$

CKT diagram :-



Block diagram :-

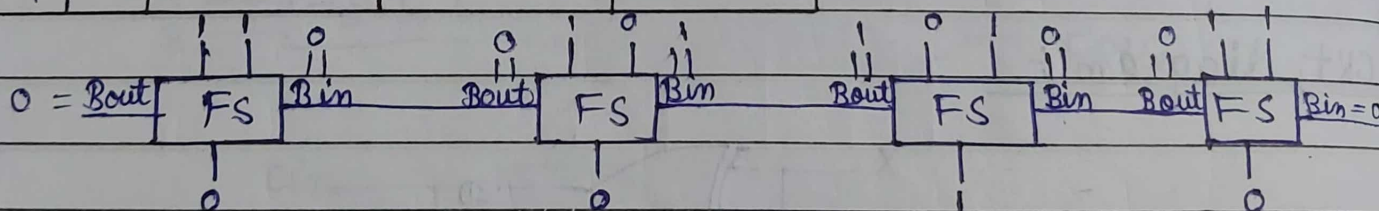


Full subtractor :-

It has 3 i/p x, y & B_{in} , which denote minuend, subtrahend & borrow resp. The 2 o/p D & B_{out} represent the diff. & o/p borrow resp.

Truth table :-

I/P			O/P	
A	B	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



K-map for D

A \ Bc	00	01	11	10
0		1		1
1	1		1	

K-map for Bout

A \ Bc	00	01	11	10
0		1	1	1
1				

Spiral

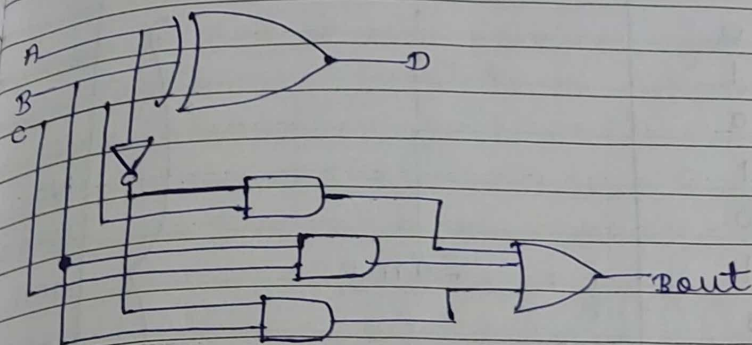
$$D = A\bar{B}\bar{C} + \bar{A}B\bar{C} + A\bar{B}C + \bar{A}B\bar{C}$$

$$= A(\bar{B}\bar{C} + B\bar{C}) + \bar{A}(\bar{B}C + B\bar{C})$$

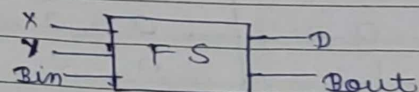
$$= A(\bar{B} \oplus C) + \bar{A}(B \oplus C) \Rightarrow A \oplus B \oplus C$$

$$Bout = \bar{A}C + \bar{A}B + BC$$

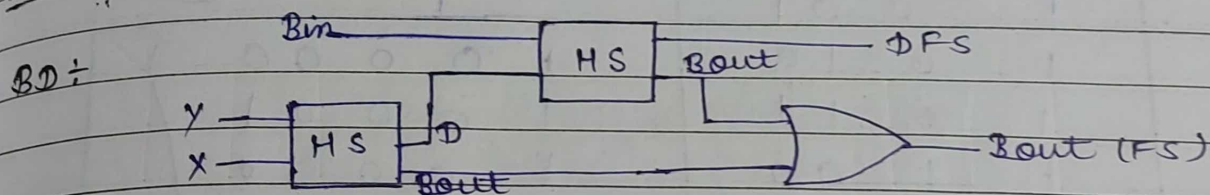
circuit diagram



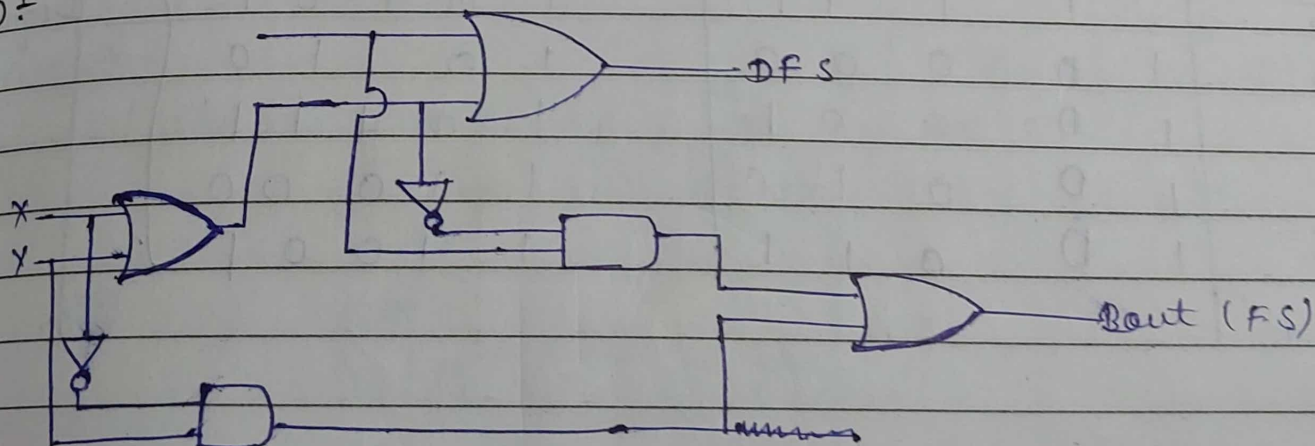
Block diagram



Full subtractor using 2 half subtractor:



CD



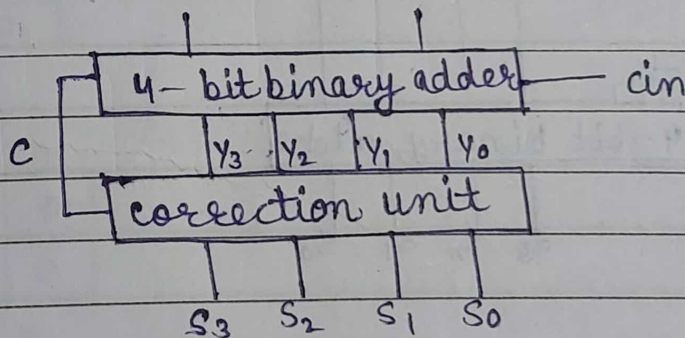
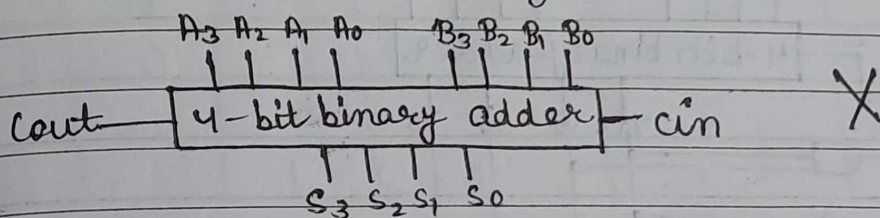
BCD adder

Decimal Digit	uncorrected BCD Sum					corrected BCD Sum					
	C	Y ₃	Y ₂	Y ₁	Y ₀	Cout	S ₃	S ₂	S ₁	S ₀	
0		0	0	0	0			1			No correction required
1		0	0	0	1			1			
2		0	0	1	0			1			
3		0	0	1	1			1			
4		0	1	0	0			1			
5		0	1	0	1		Same				
6		0	1	1	0			1			
7		0	1	1	1			1			
8		1		0	0	0					
9		1		0	0	1					
10		1		0	1	0	1	0	0	0	Correction required
11		1		0	1	1	1	0	0	0	
12		1		1	0	0	1	0	0	1	
13		1		1	0	1	1	0	0	1	
14		1		1	1	0	1	0	1	0	
15		1		1	1	1	1	0	1	0	
16	1	0		0	0	0	1	0	1	1	
17	1	0		0	0	1	1	0	1	1	
18	1	0		0	1	0	1	1	0	0	
19	1	0		0	1	1	1	1	0	0	

BCD adder is a combinational logic ckt that adds 2 BCD no. in parallel & produces a valid BCD no. as o/p.

It consists of a :-

- 4 bit binary adder for initial addition.
- Logic ckt to detect whether the sum is greater than 9 (1001_2).
- 4 bit adder to add 6 (0110_2) which the sum if $\text{sum} > 9$ or carry $c = 1$.



Case-I

When range is for 10 to 15.

y_3	y_2	y_1	y_0	00	01	11	10
00	00						
00	01						
11	00	1		1		1	1
10	01					1	1

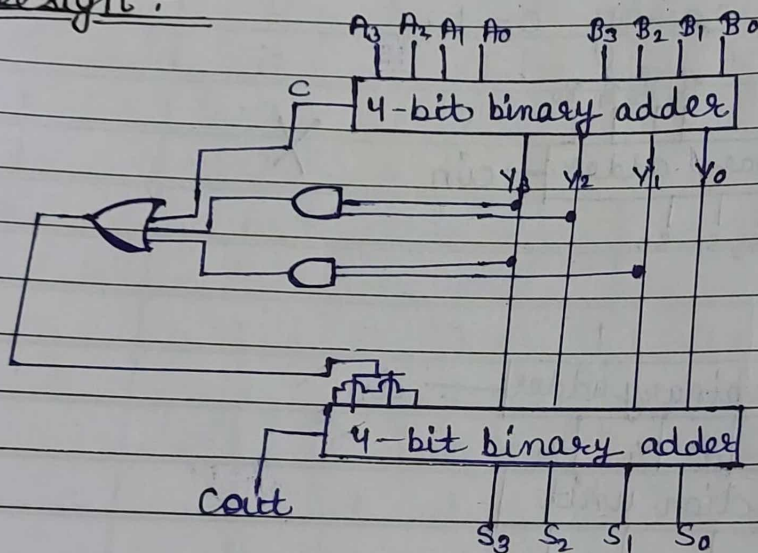
$$= Y_3 Y_2 + Y_3 Y_1$$

Case-②

When range is for 16-19.

In this case, we can signal the CRT when $C=1$. So,
overall formula for adding 6 = $Y_3 Y_2 + Y_3 Y_1 + C$

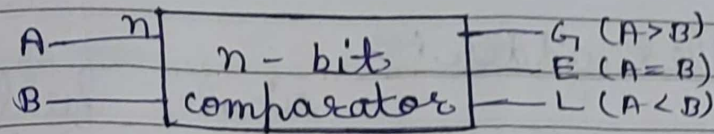
Design :-



Magnitude Comparators :-

A mag. comparators is a combinational ckt that compares 2 numbers A, B & determine there relative magnitude. The outcome of comparators is specified by 3 binary variables that indicates $A > B, A = B, A < B$.

Spiral



Q) 1-bit comparator:-

It is used to compare ~~2~~ bits
2 one bit of A & B

Truth table:-

A	B	$G (A > B)$	$E (A = B)$	$L (A < B)$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

G:-

A \ B	0	1
0	0	0
1	1	0

$$G = A \cdot \bar{B}$$

E:-

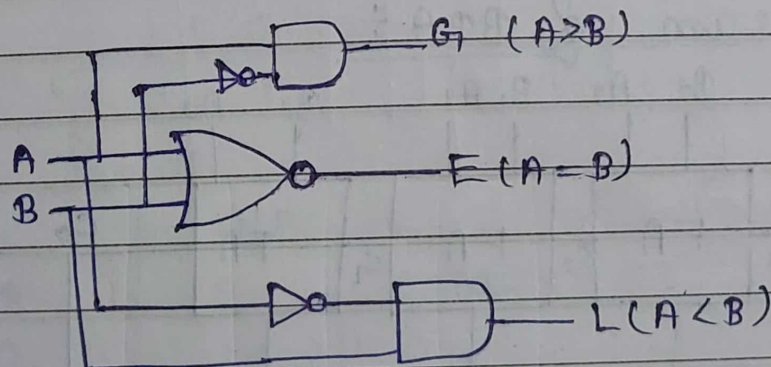
A \ B	0	1
0	1	0
1	0	1

$$E = \bar{A}\bar{B} + AB \quad (A \odot B)$$

L:-

A \ B	0	1
0	0	1
1	0	0

$$L = \bar{A}B$$



Binary parallel adder:

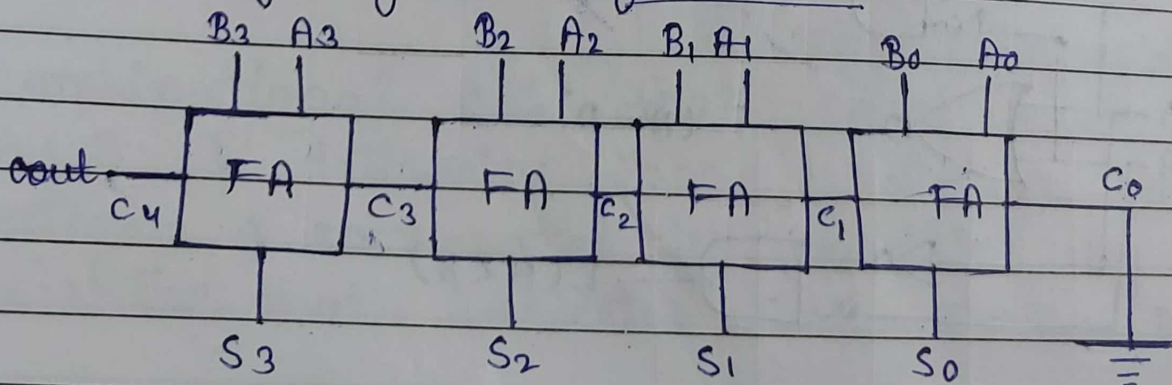
• A single full adder performs the addition of two one bit numbers and an i/p carry. But a parallel adder is a digital ckt capable of finding the arithmetic sum of 2 binary numbers i.e. greater than one bit in length by operating on corresponding pairs of bits in parallel.

• It consists of full adders connected in a chain, where the o/p carry from each full adder is connected to the carry i/p of next higher order full adder in the chain.

• A 'n' bit parallel adder requires 'n' full adders to perform the operation.

• So, for 2 bit no., two full adders are needed while for 4 bit no., 4 full adders are needed and so on.

Block diagram of BPA:



$n \times \Delta t$
delay time

Spiral

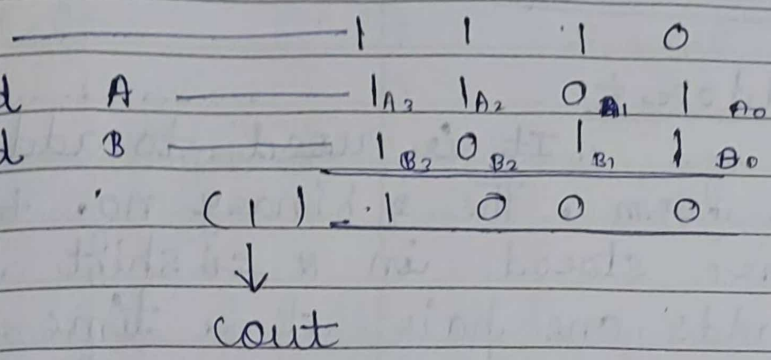
Significant position

4th 3rd 2nd 1st

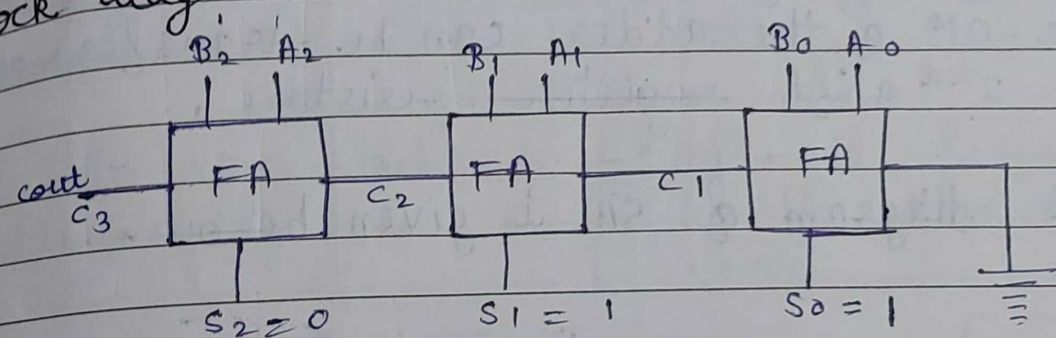
i/p carry

Addend word

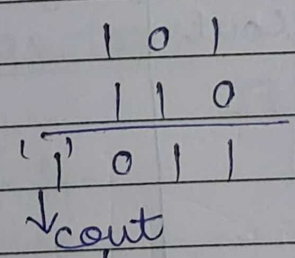
Addend word



Q. Implement the binary parallel adder, which adds 101 & 110 and then find the OP with block diagram.



A = 101, B = 110



I/P		O/P	
A	B	S	cout
1	0	1 (S ₀)	0
0	1	1 (S ₁)	0
1	1	0 (S ₂)	1

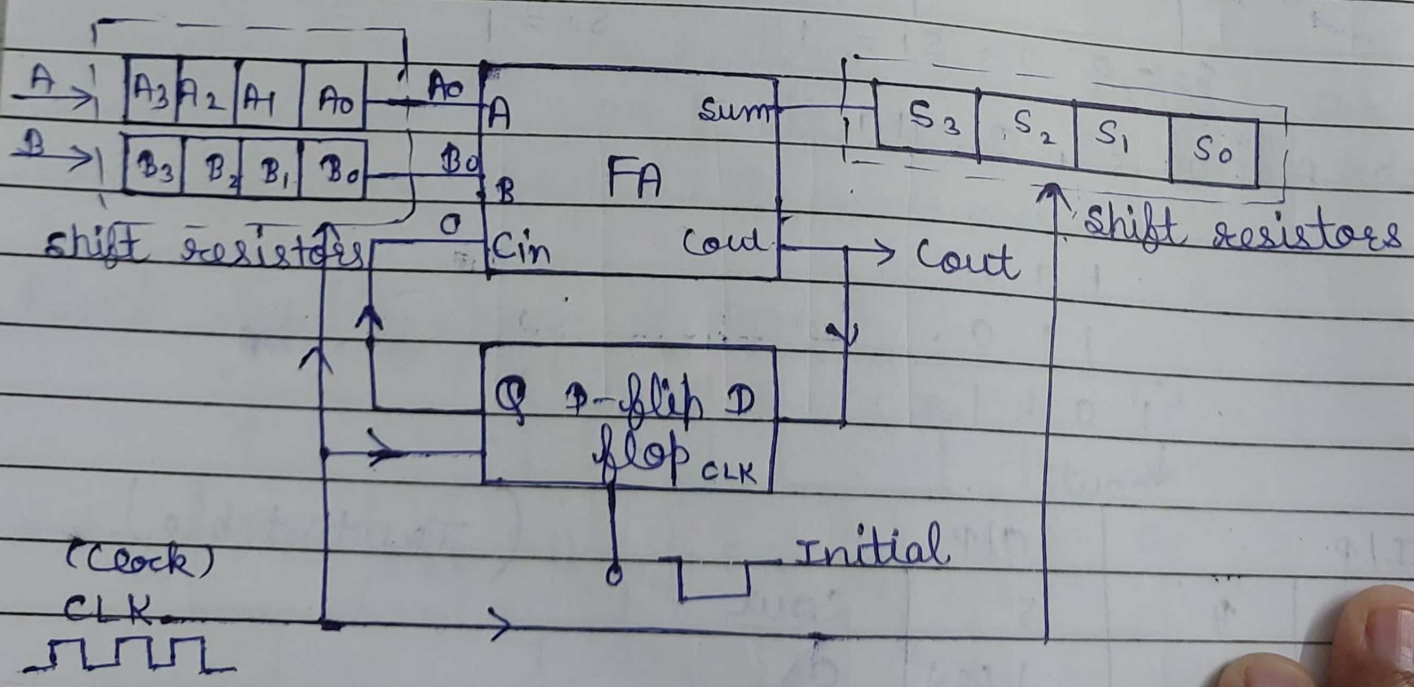
(Truth table)

So, $Sum = 011$
 $cout = 1$

Serial adder :-

It is used to add a binary no. in serial form. The 2 binary no. serially are stored in a shift register. The ckt adds one pair at a time with the help of 1 full adder. The carry out from the full adder is applied to a e.p. flip-flop, the out of which is then used as a carry in for the next pair of significant bits. The sum bit 's' from the out of the adder can be transferred into the 3rd shift register.

The block diagram of SA is given below.



Comparison of parallel and serial adder:

Parallel

- It is faster than serial adder.
- It requires a no. of full adders equivalent to the no. of bits in binary bits.
- Basically, it is a combinational CKT.
- All bits are added simultaneously.

Serial

- It is less fast due to serial addition of bits.
- It requires only 1 full adder.
- It is a sequential CKT.
- Add. is performed bit by bit starting from LSB.

Magnitude comparator:

(2) 2-bit comparator:

It is used to compare 2-bits of both A & B.

Truth table:

	A_1	A_0	B_1	B_0	$G_1 (A > B)$	$E (A = B)$	$L (A < B)$
0	0	0	0	0		1	
1	0	0	0	1			
2	0	0	1	0			1
3	0	0	1	1			1
4	0	1	0	0	1		1
5	0	1	0	1		1	
6	0	1	1	0			
7	0	1	1	1			1
8	1	0	0	0	1		1
9	1	0	0	1	1		
10	1	0	1	0		1	
11	1	0	1	1			
12	1	1	0	0	1		1
13	1	1	0	1	1		
14	1	1	1	0	1		
15	1	1	1	1		1	

G_1 :

$A_1 A_0 \backslash B_1 B_0$	00	01	11	10
00				
01	1			
11	1	1		1
10	1	1		

$$G_1 = A_1 B_1 + \overline{B_1} \overline{B_0} A_0 + A_1 A_0 \overline{B_0}$$

E :

$A_1 A_0 \backslash B_1 B_0$	00	01	11	10
00	1			
01		1		
11			1	
10				1

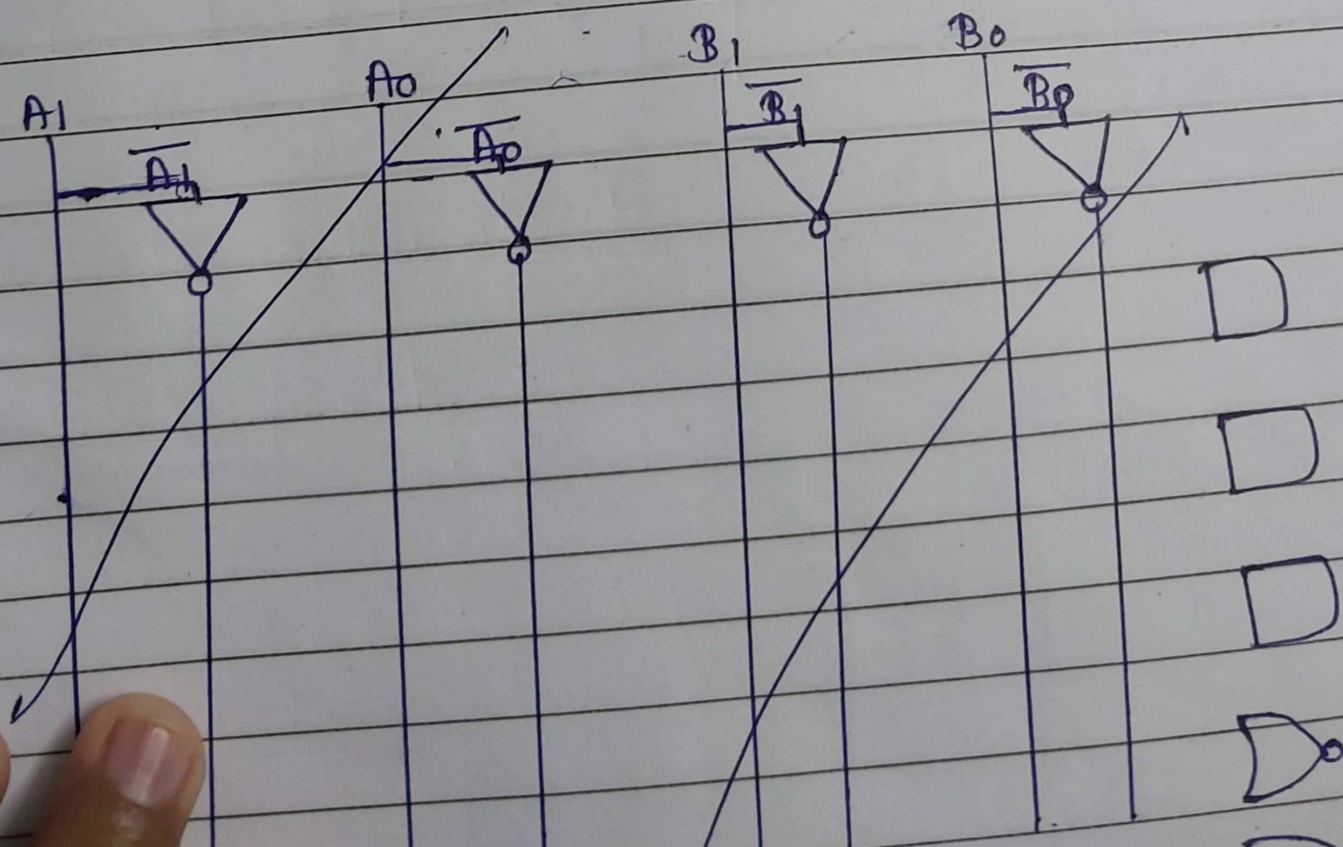
$$E = \overline{A_1} \overline{A_0} \overline{B_1} \overline{B_0} + \overline{A_1} A_0 \overline{B_1} B_0 + A_1 \overline{A_0} B_1 B_0 + A_1 A_0 B_1 \overline{B_0}$$

$$E = (A_1 \odot B_1) \cdot (A_0 \odot B_0)$$

L

$A_1 A_0 \backslash B_1 B_0$	00	01	11	10
00		1	1	1
01			1	1
11				
10			1	

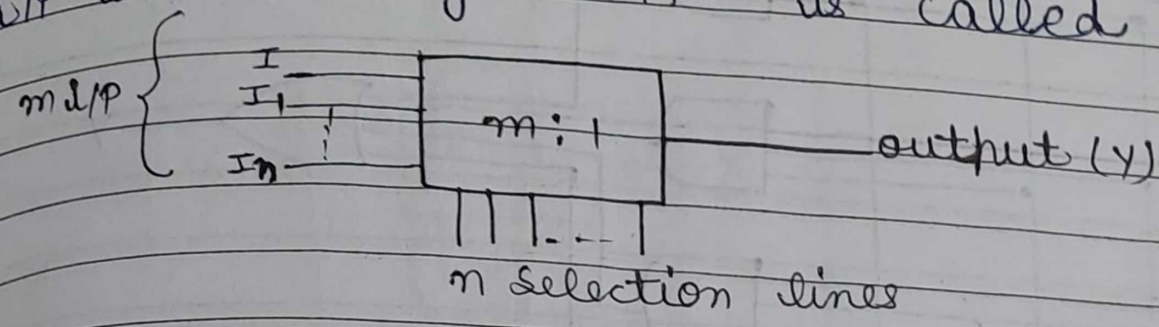
$$L = \overline{A_1} B_1 + \overline{A_1} A_0 B_0 + B_1 B_0 \overline{A_0}$$



Multiplexer:

The term multiplex means "many into one". It means transmitting large information or i/p to a single o/p.

The device which is used to convert many i/p into a single o/p is called MUX.

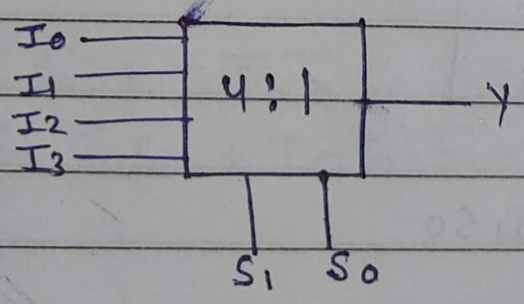


4:1 MUX:

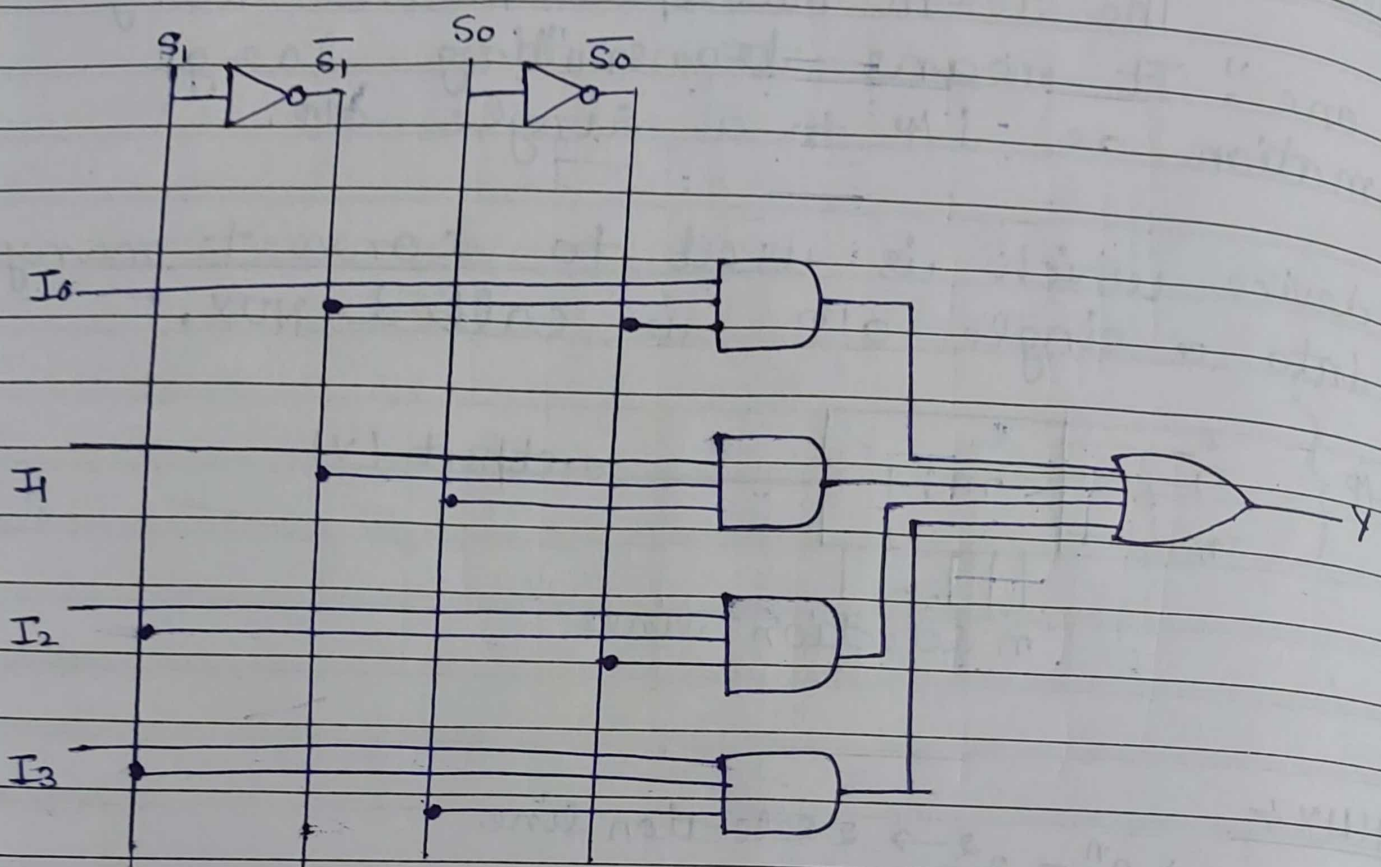
$\rightarrow 2^n = 2^2 \rightarrow 2 \text{ selection line}$

Truth table:

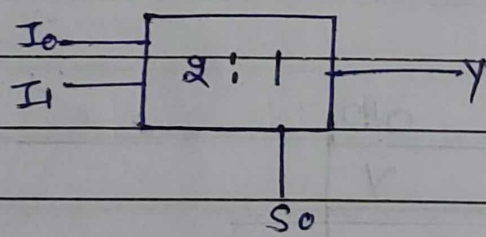
Selection i/p		o/p γ
S_1	S_0	
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



$$Y = I_0 \bar{S}_0 \bar{S}_1 + I_1 \bar{S}_1 S_0 + I_2 S_1 \bar{S}_0 + I_3 S_1 S_0$$



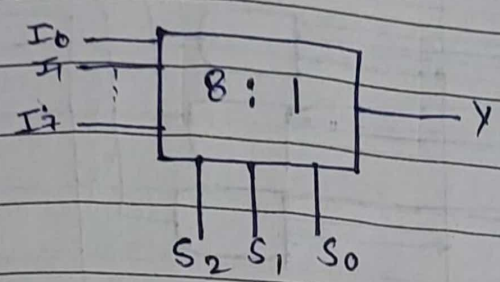
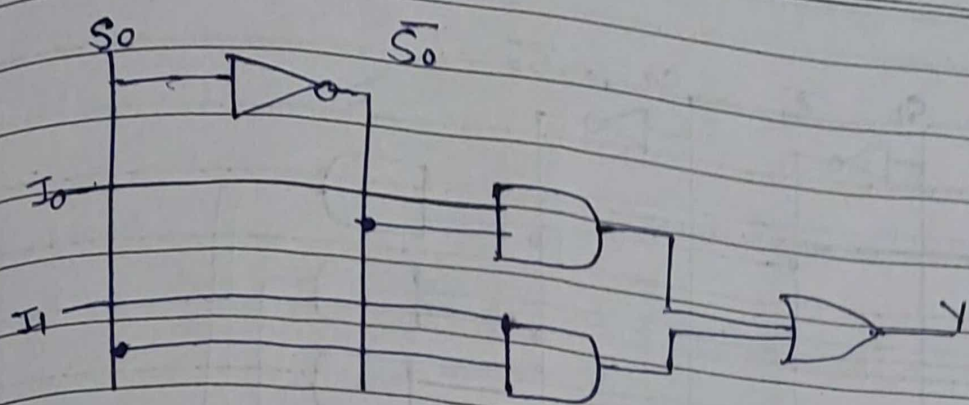
2:1



Truth Table:

S_0	Y
0	I_0
1	I_1

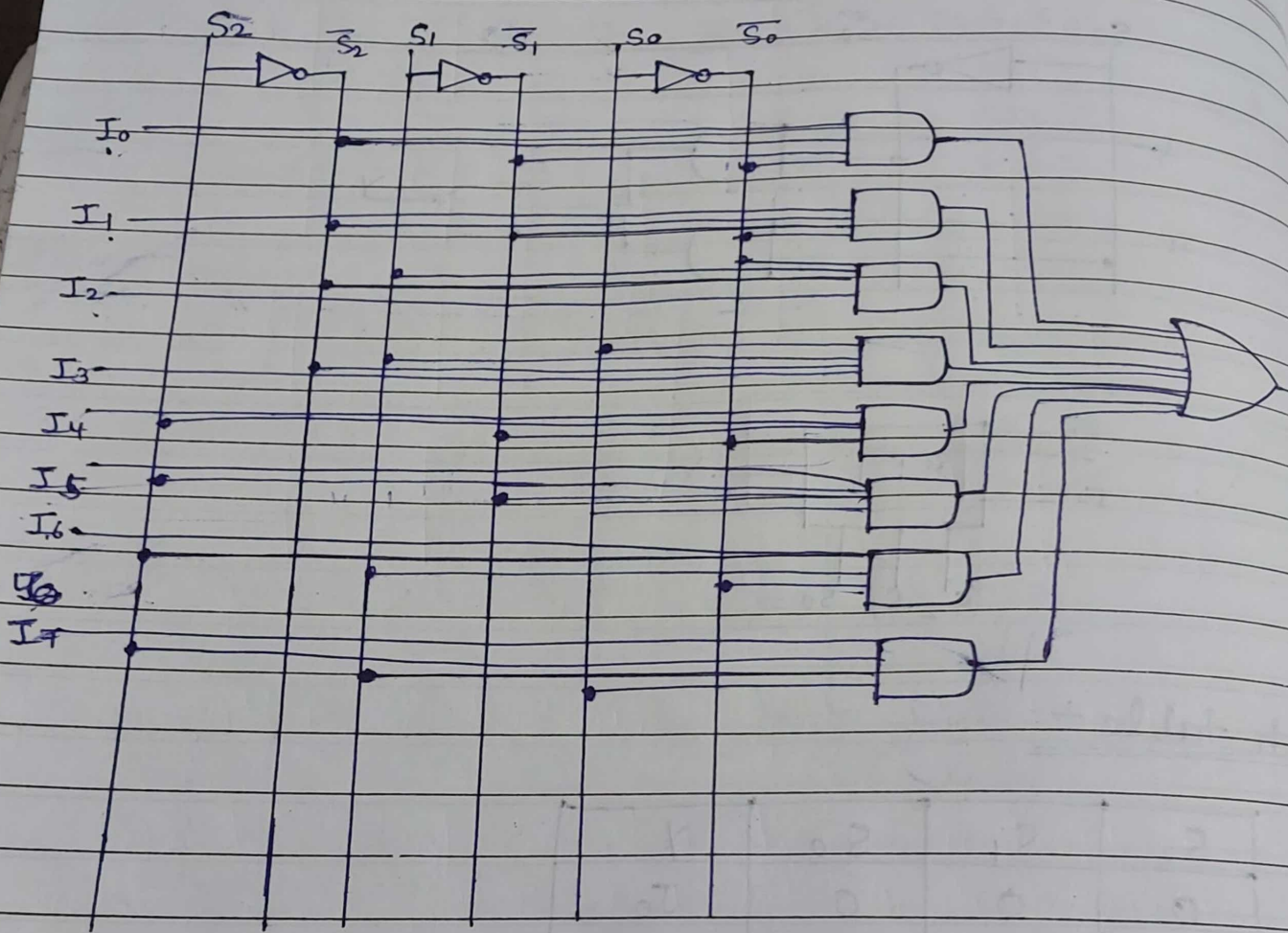
$$Y = I_0 \bar{S}_0 + I_1 S_0$$



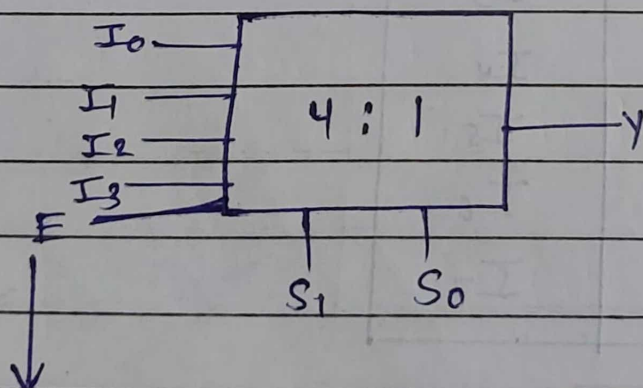
Truth table +

S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

$$Y = I_0 \overline{S_2} \overline{S_1} \overline{S_0} + I_1 \overline{S_2} \overline{S_1} S_0 + I_2 \overline{S_2} S_1 \overline{S_0} + I_3 \overline{S_2} S_1 S_0 + I_4 S_2 \overline{S_1} \overline{S_0} + I_5 S_2 \overline{S_1} S_0 + I_6 S_2 S_1 \overline{S_0} + I_7 S_2 S_1 S_0$$



MUX with enable I/P :-



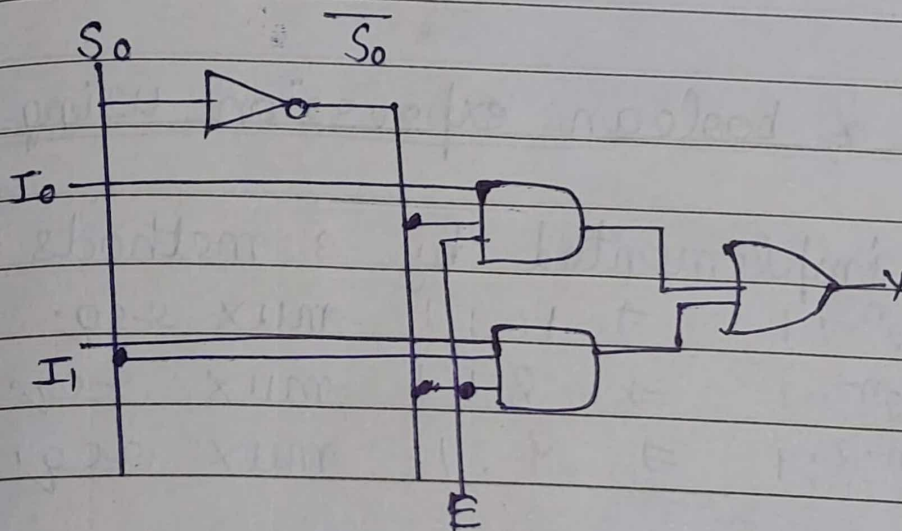
Active low $\Rightarrow E = 0$
Active high $\Rightarrow E = 1$

Truth table:

E	S ₁	S ₀	Y
0	X	X	0
1	0	0	I ₀
1	0	1	I ₁
1	1	0	I ₂
1	1	1	I ₃

$$Y = I_0 \bar{S}_1 \bar{S}_0 + I_1 \bar{S}_1 S_0 + I_2 S_1 \bar{S}_0 + I_3 S_1 S_0$$

2:1

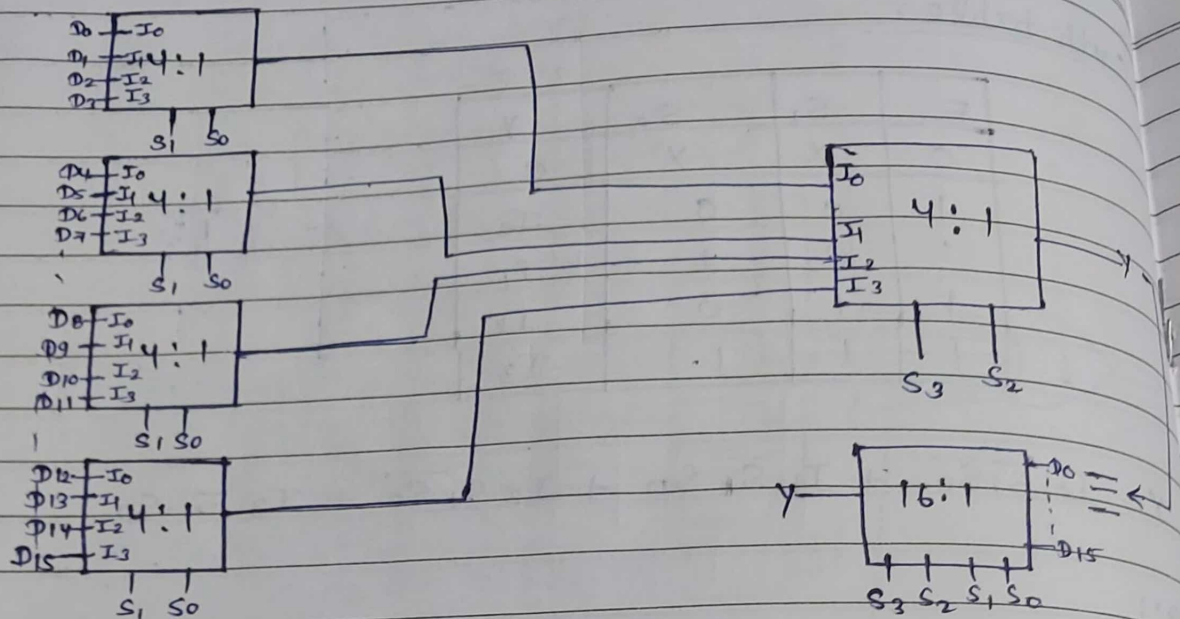


Higher order MUX:

16:1 MUX:

↓
2⁴ → 4 selection line

By 4:1



Implementation of boolean expression using MUX:

This can be implemented by 3 methods:

- (1) Type 0 $\rightarrow 2^n : 1 \Rightarrow 16 : 1$ mux req.
- (2) Type 1 $\rightarrow 2^{n-1} : 1 \Rightarrow 8 : 1$ mux req.
- (3) Type 2 $\rightarrow 2^{n-2} : 1 \Rightarrow 4 : 1$ mux req.

Q. $f(A, B, C, D) = \sum m(0, 1, 2, 5, 8, 13, 14)$ implement this boolean expression using type (0, 1, 2) method.

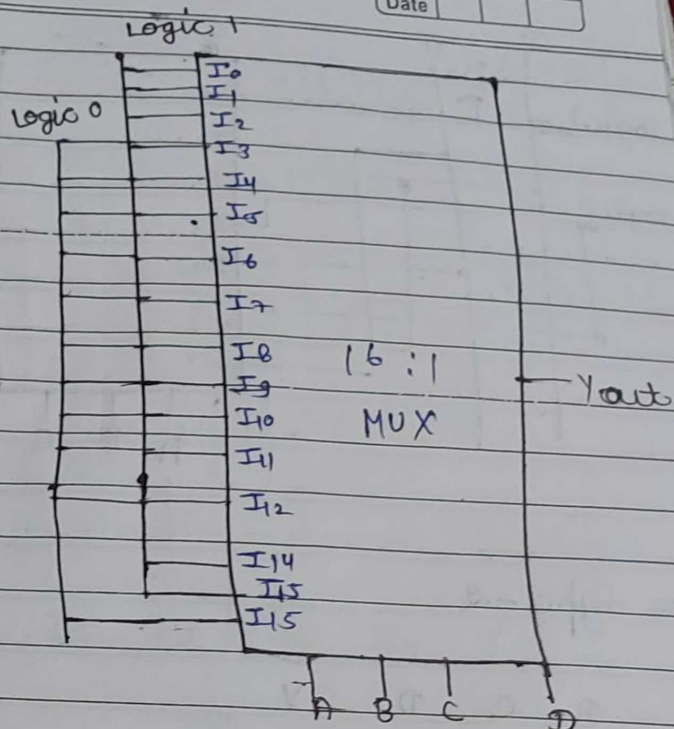
for type 0 - 4 variables

$$\text{Req. mux} = 2^n : 1$$

$$= 2^4 : 1$$

$$= 16 : 1 \text{ mux}$$

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

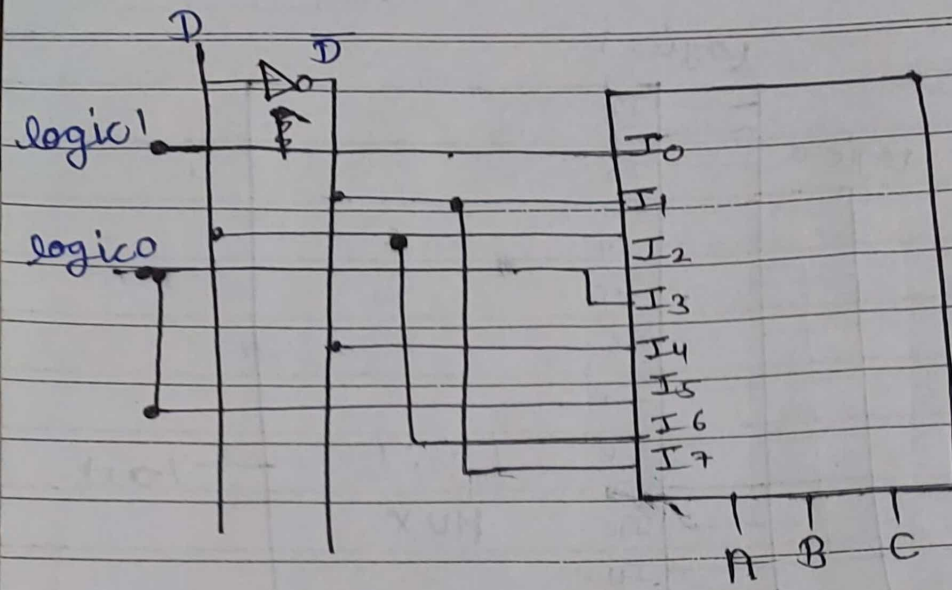


for type -1

Implementation table

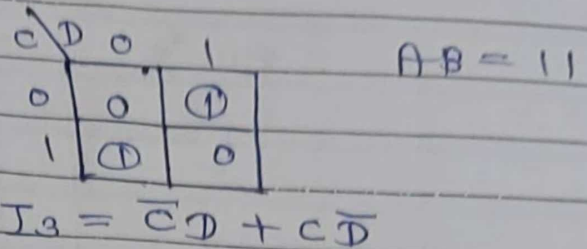
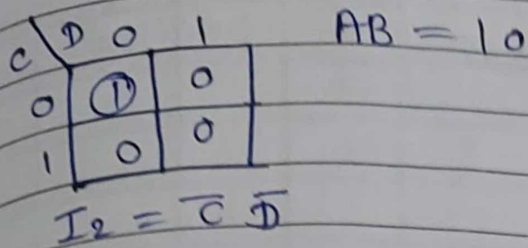
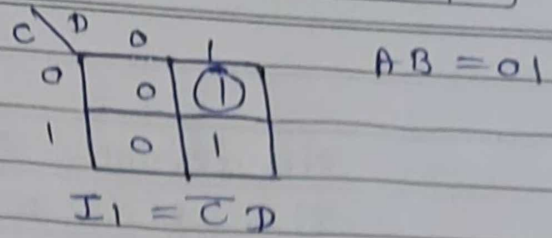
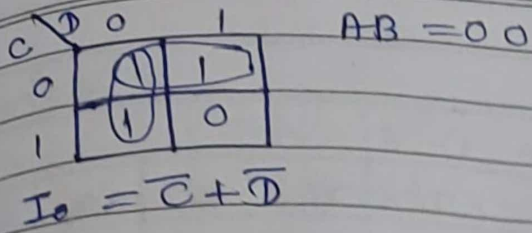
	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7		A	B	C	Y
$\overline{D}$	0	2	4	6	8	10	12	14		0	0	0	1
D	1	3	5	7	9	11	13	15		0	0	1	$\overline{D}$
	1	$\overline{D}$	D	0	$\overline{D}$	0	D	$\overline{D}$		0	1	1	0
										1	0	0	$\overline{D}$
										1	0	1	0
										1	1	0	$\overline{D}$
										1	1	1	D

Spiral



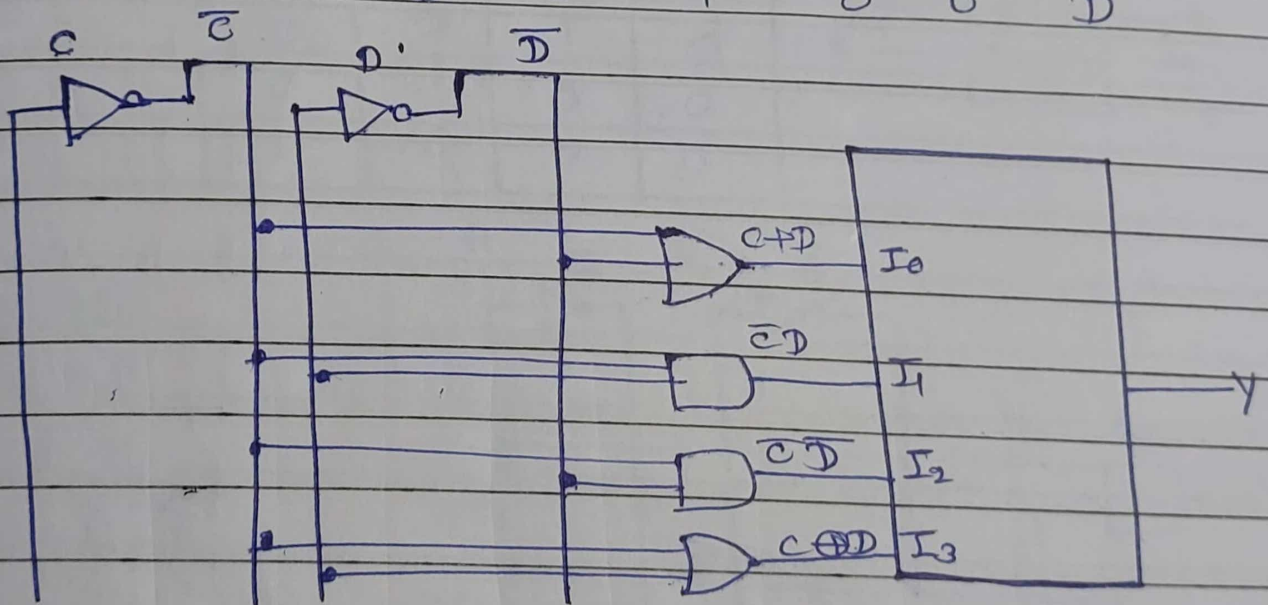
for type - 2

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0



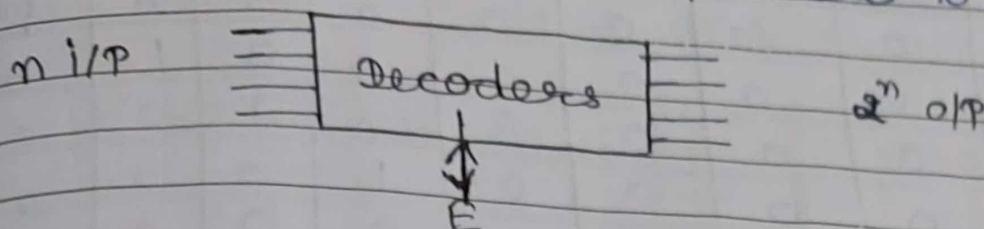
Implementation table:

	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
d	0	2	4	6	8	10	12	14
$\bar{d}$	1	3	5	7	9	11	13	15
	1	d	$\bar{d}$	0	1	0	0	d



Decoder:

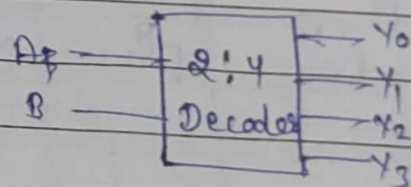
It is a multi i/p, multi o/p logic ckt which decodes $n \times 2^n$ possible o/p.



where $E=0$, decoder is disabled
 $E=1$, decoder is enabled.

(i) 2 to 4 decoder:
 $2^2 = 4$

A	B	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

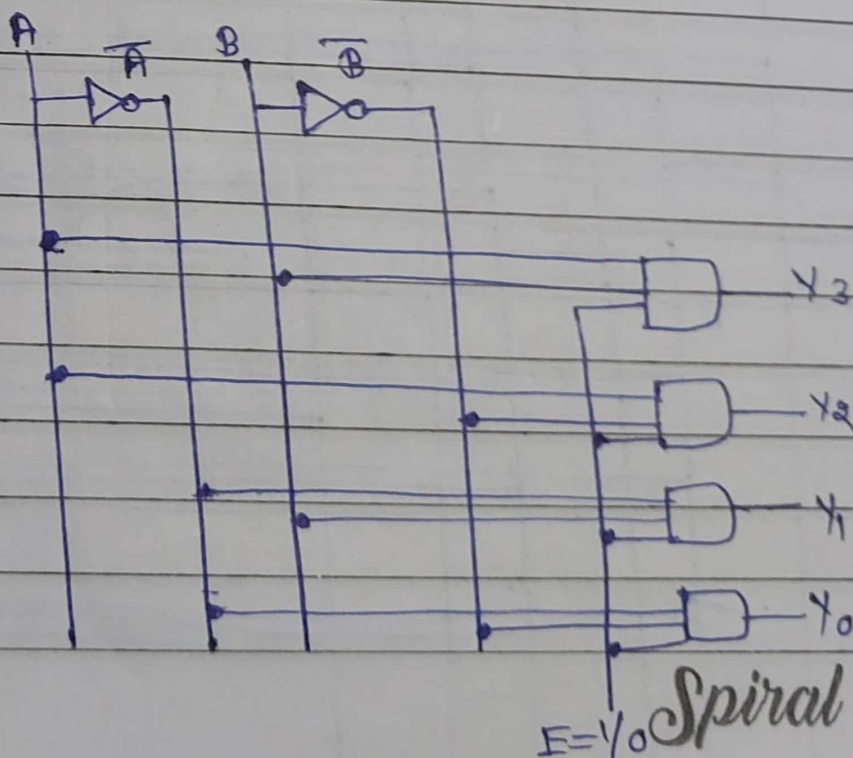


$$Y_3 = AB$$

$$Y_2 = A\bar{B}$$

$$Y_1 = \bar{A}B$$

$$Y_0 = \bar{A}\bar{B}$$

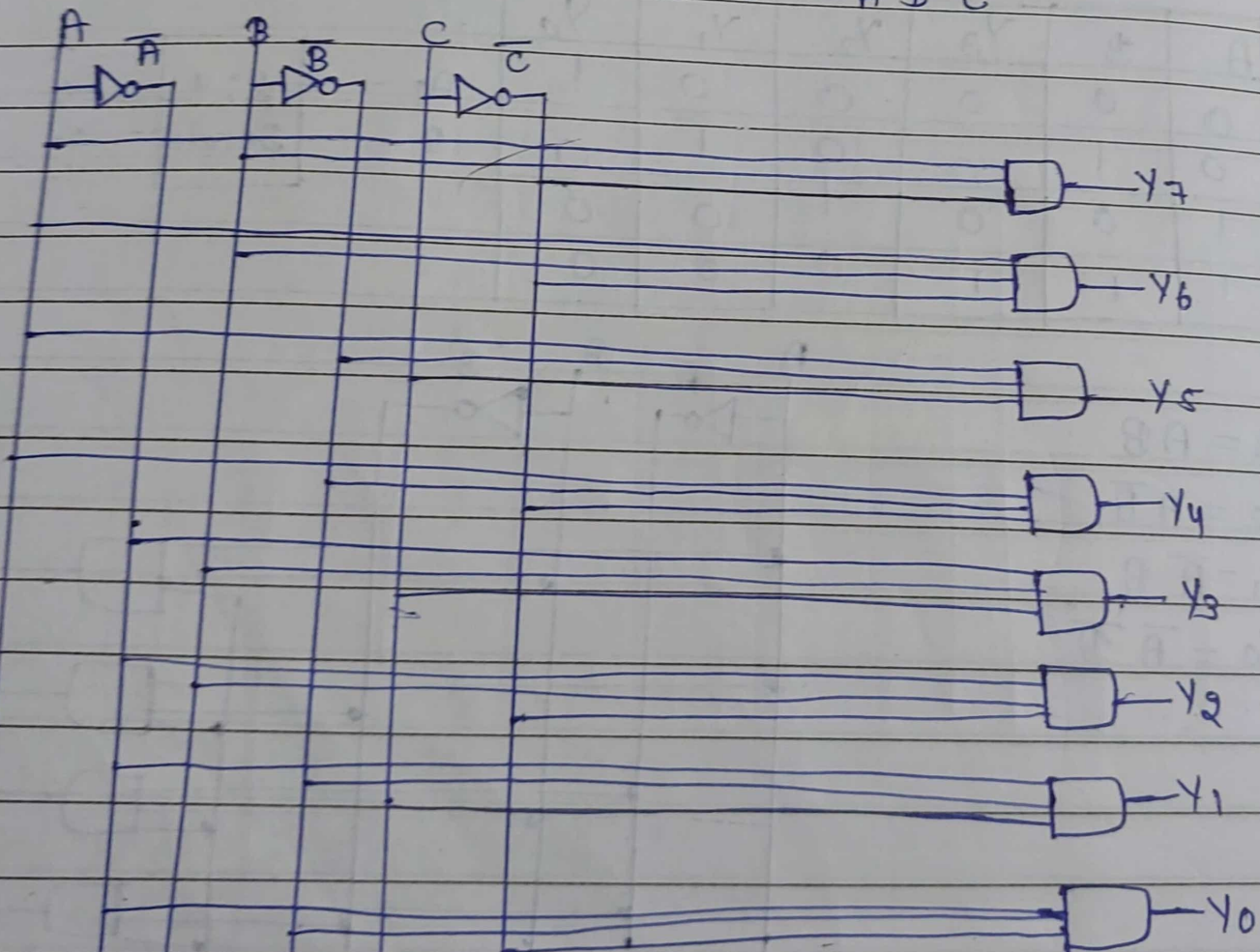


$E = Y_0$ *Spiral*

(2) 3 to 8 decoder ÷

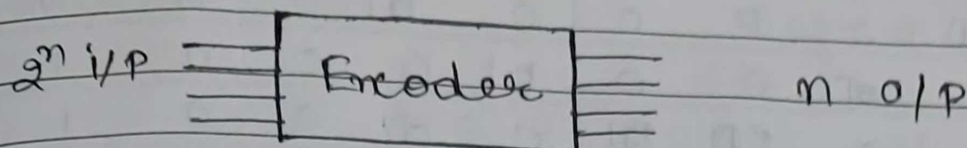
	A	B	C	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	1	1
2	0	1	0	0	0	0	0	0	1	0	0
3	0	1	1	0	0	0	0	1	0	0	0
4	1	0	0	0	0	0	1	0	0	0	0
5	1	0	1	0	0	1	0	0	0	0	0
6	1	1	0	0	1	0	0	0	0	0	0
7	1	1	1	1	0	0	0	0	0	0	0

$Y_7 = ABC$, $Y_6 = A\bar{B}C$, $Y_5 = A\bar{B}\bar{C}$, $Y_4 = A\bar{B}C$, $Y_3 = \bar{A}BC$
 $Y_2 = \bar{A}B\bar{C}$, $Y_1 = \bar{A}B C$, $Y_0 = \bar{A}\bar{B}\bar{C}$



Encoder :-

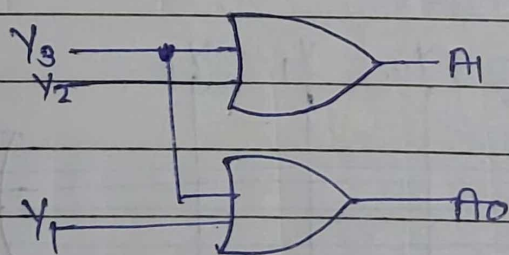
It is a combinational CKT that performs the reverse operation of decoder. It has max. of 2^n i/p lines and n o/p lines.



(i) 4:2 encoder :-

	i/p				o/p	
	Y_3	Y_2	Y_1	Y_0	A_1	A_0
0	0	0	0	1	0	0
1	0	0	1	0	0	1
2	0	1	0	0	1	0
3	1	0	0	0	1	1

$$A_1 = Y_2 + Y_3 \quad , \quad A_0 = Y_1 + Y_3$$



(ii) 8:3 encoder :-

Truth table :-

i IP

	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	A_2	A_1	A_0
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0	0

$$A_2 = Y_4 + Y_5 + Y_6 + Y_7, A_1 = Y_2 + Y_3 + Y_6 + Y_7$$

$$A_0 = Y_1 + Y_3 + Y_5 + Y_7$$

$$A_2 = Y_4 + Y_5 + Y_6 + Y_7$$

$$A_2 = Y_4 + Y_5 + Y_6 + Y_7, A_1 = Y_3 + Y_2 + Y_7 + Y_6$$

$$A_0 = Y_1 + Y_3 + Y_5 + Y_7$$

